

Module 3: Permutation Feature Importance

Instructional Hours: 6

Module Overview

In this module you will learn about Permutation Feature Importance (PFI) as a technique for interpreting machine learning models. You will explore the concept, its role in explainable AI, and how it is used to rank feature importance across various models. You will also cover practical applications, advantages, limitations, and implementation in Python.

Learning Outcomes

By the end of this module, you will be able to:

1. Explain the intuition behind using permutation feature importance to rank the importance of features
2. Discuss how feature importances are calculated using permutation feature importance
3. Create code to implement permutation feature importance to explain an AI model
4. Justify the use of permutation feature importance in explaining a AI model

Learning Activities/task List

1. Review Introductory notes on permutation feature importance
2. Contribute to a discussion on permutation feature importance
3. Review selected readings on permutation feature importance
4. Complete a quiz on key permutation feature importance
5. Implement Python Code to calculate and plot permutation feature importance
6. Complete a final quiz on permutation feature importance

Introduction To Permutation Feature Importance

Permutation Feature Importance (PFI) is a model-agnostic method used to evaluate the effect of each of the features on the performance of a machine learning model. It involves shuffling the values of a specific feature to disrupt its relationship with the target variable and measuring the drop in model performance (for instance in accuracy or root mean squared error). The greater the drop in performance, the more important the feature is considered to be. As an example, assume that for some variable, the value ' a ' is associated with an output value ' c ', then after shuffling, we may end up with different relationship such as ' b ' being associated with ' c ' as illustrated below.

$$a \longrightarrow c$$

After shuffling

$$b \rightarrow c$$

The second relationship is spurious and like saying 2 squared is 6. The drop in performance will be as a result of the model saying that the second relationship is incorrect. That is, the newly assigned value (b) cannot predict c .

PFI is important in explainable AI because it provides an intuitive way to explain the contribution of features to model decisions, making it easier for stakeholders to understand how the model works. By showing the importance of each feature, PFI helps to interpret the model's behavior, especially in complex models like random forests or gradient boosting. One of the main advantage of PFI is that it can be applied to any machine learning model, whether it's a decision tree, neural network, or support vector machine.

How Permutation Feature Importance Works

The following is the sequence of steps you use to implement explainability using permutation feature importance algorithm

1. **Train the Model**

Split the data into the training and test set and train the model on the test set.

2. **Get the Baseline Performance**

You get the model's performance on a test set, using a chosen metric such as accuracy, F1 score, or mean squared error.

3. **Feature Permutation**

Shuffle the values of a single attribute across all samples in the test set. To achieve this, for each of the examples in the test set, add the value that is assigned to the feature into an array. Once all the values have been copied

into the array, shuffle the items in the array. For each training example, assign the attribute of interest the value that has the same position as itself in the shuffled array.

4. **Re-evaluating the Model Performance**

You then measure the model's performance with the permuted feature using the test set.

5. **Calculate Importance**

The importance of the feature is the difference between the baseline performance and the performance after permutation. Larger differences indicate more important features.

Steps 3 to 5 are repeated for all the features.

Strengths of PFI

Model-Agnostic - This technique works with any machine learning model without requiring internal access to the model.

Easy to Implement - It can be easily calculated by permuting features and evaluating performance.

Interpretability - It provides clear, quantitative information on feature relevance that is understandable to non-experts.

Limitations of PFI

Feature Correlation - It can produce misleading results in the presence of correlated features. If two features are highly correlated, permuting one may not reduce model performance significantly, even though both features are important.

Computational Cost - Shuffling features and re-evaluating model performance for each feature can be computationally expensive, especially with large datasets or complex models.

Dependence on Model Performance Metric - The importance values depend on the performance metric used, which might not always align with business needs or domain-specific goals.

Permutation Features Importances Usage Examples

Reading Material 1

To learn about practical examples in the use of permutation feature importance read the following material

Permutation Feature Importance

Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.), URL: <https://christophm.github.io/interpretable-ml-book/feature-importance.html>

Implementation Of Permutation Feature Importance Using Python

You can implement permutation feature importance using python. You should use the permutation_importance class from the sklearn.inspection package. This is illustrated by the below:

```
#import the necessary packages
from sklearn.ensemble import RandomForestRegressor as RFR
from sklearn.metrics import accuracy_score
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error as mse, r2_score
from sklearn.datasets import load_diabetes
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#Load the data
data=load_diabetes()
#print(data.DESCR)
#define the data
x=data.data
y=data.target
print(x.shape)
print(y.shape)
#Get the columns of the data
columns=data.feature_names.copy()
print(columns)

#Put the data in a dataframe for easier visualization
import pandas as pd
df = pd.DataFrame(x, columns=columns)
df['target']
#show the correlation
corr_matrix = df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
linewidths=0.5)
# Add a title
plt.title('Correlation Heatmap')
```

```

plt.show()
# split the data into training and test set
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.4, random_state=42)
# fit the model
model=RFR(n_estimators=100, random_state=42)
model = model.fit(x_train, y_train)
#Show the features importances calculated by the random forest
technique by default
for col, val in zip(columns,model.feature_importances_):
    print("{}: {:.2f}, ".format(col, val), end="")
# Make predictions
predictions=model.predict(x_test)
for val in predictions[0:10]:
    print("{}: {:.0f}\t".format(val), end="")
print()
for val in y_test[0:10]:
    print("{}: {:.0f}\t".format(val), end="")
#view RMSE values and R-Squared values
import numpy as np
rms=np.sqrt(mse(y_test,predictions))
print("RMSE: {:.2f}".format(rms))
r2=model.score(x_test, y_test)
print("R2: {:.2f}".format(r2))

# Apply Permutation Feature Importance
result = permutation_importance(model, x_test, y_test, n_repeats=10,
random_state=42, scoring='neg_root_mean_squared_error')
for i in range(len(result.importances_mean)):
    print(f"Feature {data.feature_names[i]}:
{result.importances_mean[i]:.3f} +/- {result.importances_std[i]:.3f}")
dfSummary=pd.DataFrame({'Variable':columns,
'importance':result.importances_mean, 'coeff_lb':-
result.importances_std[i], 'coeff_ub':result.importances_std[i]})
dfSummary.head(10)
#Sort the values
dfSorted=dfSummary.sort_values(by="importance", ascending=True)
dfSorted.head(11)
# Plotting the bar chart
plt.figure(figsize=(8,6))
plt.barh(dfSorted['Variable'], dfSorted['importance'],
color='skyblue', height=0.2)
#labels and title
#plt.xlabel('importance')
#plt.ylabel('variable')

```

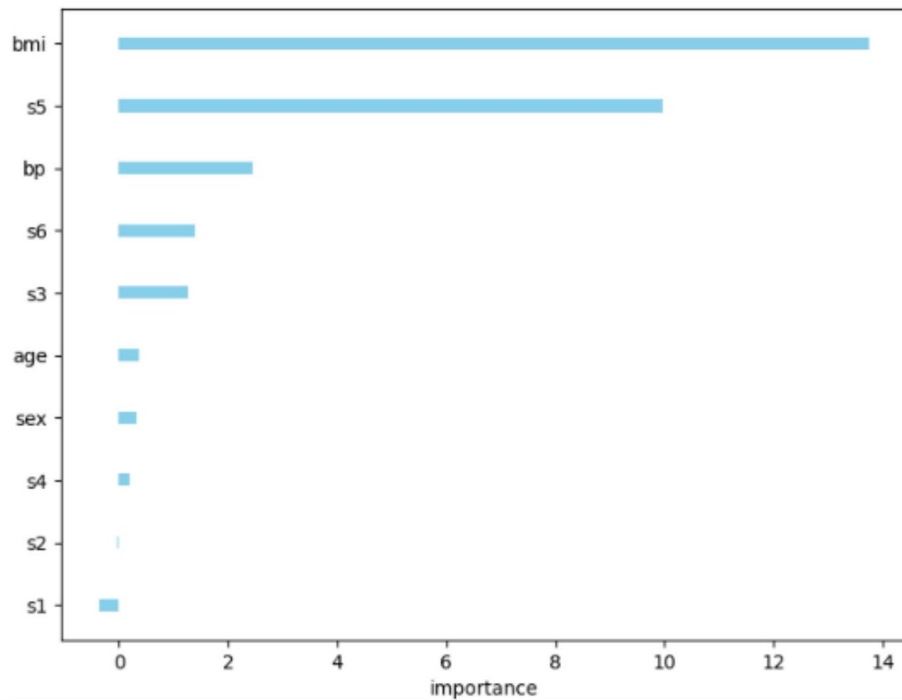


Figure 3.1: Feature importances as calculated by permutation feature importance

Figure 3.1 shows the permutation feature importance as calculated using the PFI algorithm a random forest classifier for the diabetes dataset. The measure of performance is the root mean squared error (RMSE). The results show that shuffling the bmi attribute results in the RMSE increasing by around 14. Randomizing the s1 and s2 variables actually results in reduction in the RMSE.